Django Forms — Complete Notes (English)

1. What Are Django Forms?

Django Forms is a built-in framework that helps developers collect user input, generate HTML form fields, validate submitted data, convert values into Python data types, and process form submissions.

Common Uses

- User registration and login
- Contact forms
- Search and filter forms
- Profile updates
- Product creation
- File uploads
- Data entry and editing
- Bulk data entry using formsets

Advantages of Django Forms

1. Reduces the amount of HTML and Python code required.
2. Provides built-in field validation.
3. Converts input data into appropriate Python types.
4. Displays validation errors.
5. Preserves submitted values when validation fails.
6. Supports custom validation.
7. Integrates with Django models through `ModelForm`.
8. Helps implement consistent and secure form handling.

---

2. HTML Forms

An HTML form collects data from users and submits it to a server.

Example

```
<form action="/your-name/" method="post">
    <label for="your_name">Your name:</label>

    <input
        type="text"
```

```
            id="your_name"
            name="your_name"
        >

        <button type="submit">Submit</button>
    </form>
```

Important HTML Attributes

Attribute	Purpose
`action`	Specifies the URL where the form is submitted
`method`	Specifies the HTTP method
`type`	Defines the input type
`name`	Identifies the submitted value
`id`	Identifies an HTML element
`required`	Makes an input mandator

	y in the browser
`placeholder`	Displays a hint inside an input
`enctype`	Defines how form data is encoded

The HTML `required` attribute improves browser-side validation, but server-side validation is still necessary.

---

3. GET and POST Methods

GET Method

GET is primarily used to retrieve data from a server.

```
<form action="/search/" method="get">
    <input type="text" name="q">
    <button type="submit">Search</button>
</form>
```

Example URL:

```
/search/?q=django
```

Characteristics:

- Data is included in the URL query string.
- URLs can be bookmarked and shared.
- Suitable for searching, filtering, and retrieving information.
- Should not be used for operations that change server state.
- Sensitive data should not be placed in query strings.

POST Method

POST is commonly used to submit data for processing or to perform state-changing operations.

```
<form action="/register/" method="post">
    {% csrf_token %}

    <input type="text" name="username">
    <input type="email" name="email">

    <button type="submit">Register</button>
</form>
```

Characteristics:

- Data is generally sent in the request body.
- Suitable for registration, updates, and other data submissions.
- Django CSRF protection is generally required for applicable POST requests.
- POST does not encrypt data; HTTPS protects data in transit.

GET vs POST

GET	POST
Primarily retrieves data	Submits data for processing
Data appears in the URL query string	Data is generally sent in the request body
Suitable for searches	Suitable for data submission

Should not change server state	Commonly used for state-changing operations
Query data may be stored in browser history	Request body is not normally displayed in the URL

---

4. How Django Forms Work

Django Forms primarily help developers:

1. Define form fields.
2. Generate HTML form controls.
3. Validate submitted data.
4. Convert input values into Python data types.
5. Display validation errors.
6. Process or save valid data.

Form Processing Flow

```
User
  |
  v
HTML Form
  |
  v
GET or POST Request
  |
  v
Django View
  |
```

```
      v
Form Validation
  |
  +---- Valid ----> Process or Save Data
  |
  +---- Invalid --> Display Errors
```

A typical Django form submission follows the POST-Redirect-GET pattern: process the POST request, redirect after success, and display the form again if validation fails.

---

5. Creating a Django Form

Django provides the `forms.Form` class for creating regular forms.

forms.py

```
from django import forms


class NameForm(forms.Form):
    your_name = forms.CharField(
        label="Your Name",
        max_length=100
    )
```

Explanation

- `forms.Form` : Base class for regular forms.
- `NameForm` : Custom form class.
- `your_name` : Field name.
- `CharField` : Accepts text input.
- `label` : Specifies the field label.
- `max_length=100` : Limits the text length to 100 characters.

Django uses the field definition to validate input and generate the corresponding HTML widget.

---

6. Django Form Fields

Django provides many built-in form fields.

Field	Purpose

`CharField`	Text input
`EmailField`	Validates email-like input
`IntegerField`	Integer values
`FloatField`	Floating-point values
`DecimalField`	Decimal values
`BooleanField`	Boolean values
`DateField`	Date values
`DateTimeField`	Date and time values
`TimeField`	Time values
`DurationField`	Duration values
`URLField`	URL validation

`ChoiceField`	Selects one option
`MultipleChoiceField`	Selects multiple options
`FileField`	Accepts uploaded files
`ImageField`	Accepts uploaded images
`SlugField`	Validates slug-like values
`RegexField`	Validates input using a regular expression
`UUIDField`	Validates UUID values

Example

```
from django import forms


class ProductForm(forms.Form):
```

```
    name = forms.CharField(max_length=100)

    price = forms.DecimalField(
        max_digits=10,
        decimal_places=2
    )

    quantity = forms.IntegerField(
        min_value=1
    )

    available = forms.BooleanField(
        required=False
    )

    website = forms.URLField(
        required=False
    )
```

`DecimalField` is commonly preferred for financial values because decimal arithmetic is more suitable for monetary calculations than binary floating-point arithmetic.

---

7. Common Field Arguments

Field arguments control how data is accepted, displayed, and validated.

```
from django import forms


class StudentForm(forms.Form):
    name = forms.CharField(
        label="Student Name",
        min_length=3,
        max_length=50,
        required=True,
        help_text="Enter your full name."
    )

    age = forms.IntegerField(
        min_value=1,
        max_value=100
    )
```

```
        email = forms.EmailField()
```

Important Arguments

Argument	Purpose
`label`	Defines the field label
`required`	Determines whether a value is required
`initial`	Specifies an initial display value
`help_text`	Provides additional instructions
`min_length`	Defines minimum text length
`max_length`	Defines maximum text length

`min_value`	Defines minimum numeric value
`max_value`	Defines maximum numeric value
`widget`	Defines how the field is rendered
`error_messages`	Customizes validation messages
`validators`	Adds validation functions
`disabled`	Prevents ordinary user editing of a field

Custom Error Messages

```
username = forms.CharField(
    min_length=4,
    max_length=20,
    error_messages={
```

```
            "required": "Username is required.",
            "min_length": "At least 4 characters are required.",
            "max_length": "Maximum 20 characters are allowed."
        }
    )
```

---

8. Widgets

A widget controls how a Django form field is rendered as an HTML input element.

Example

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField()

    message = forms.CharField(
        widget=forms.Textarea
    )

    password = forms.CharField(
        widget=forms.PasswordInput
    )

    birthday = forms.DateField(
        widget=forms.DateInput(
            attrs={"type": "date"}
        )
    )
```

Common Widgets

Widget	HTML Representation
`TextInput`	Text input

`EmailInput`	Email input
`NumberInput`	Number input
`PasswordInput`	Password input
`Textarea`	Multiline text area
`CheckboxInput`	Checkbox
`Select`	Dropdown
`RadioSelect`	Radio buttons
`FileInput`	File input
`DateInput`	Date input
`HiddenInput`	Hidden input

Adding HTML Attributes

```
class LoginForm(forms.Form):
    username = forms.CharField(
        widget=forms.TextInput(
            attrs={
                "class": "form-control",
                "placeholder": "Enter username"
            }
```

```
            )
        )

        password = forms.CharField(
            widget=forms.PasswordInput(
                attrs={
                    "class": "form-control",
                    "placeholder": "Enter password"
                }
            )
        )
```

The `attrs` dictionary allows you to define HTML attributes such as CSS classes, placeholders, and input IDs.

Important: A widget controls presentation. The associated form field performs data validation.

---

9. Bound and Unbound Forms

Unbound Form

An unbound form has not been bound to submitted data.

```
from django import forms


class NameForm(forms.Form):
    name = forms.CharField(max_length=100)


form = NameForm()

print(form.is_bound)
```

Output:

```
False
```

Characteristics:

- No submitted data is bound to the form.
- Commonly used when displaying a form for the first time.

- Can display initial values.
- Does not represent submitted form data.

Bound Form

A bound form has data associated with it.

```
form = NameForm({
    "name": "Faruk"
})

print(form.is_bound)
print(form.is_valid())
print(form.cleaned_data)
```

Output:

```
True
True
{'name': 'Faruk'}
```

Differences

Unbound Form	Bound Form
No data is bound	Data is bound
`is_bound` is `False`	`is_bound` is `True`
Commonly used for initial display	Commonly used to process submissions

Does not contain bound submission data	May contain submitted data

Important: `NameForm({})` is a bound form, even though the dictionary is empty. A bound form can still be invalid.

---

10. The `is_valid()` Method

The `is_valid()` method runs validation on a bound form.

It returns `True` when all validation succeeds and `False` when validation fails.

```
from django import forms


class StudentForm(forms.Form):
    name = forms.CharField(max_length=50)
    age = forms.IntegerField(min_value=1)


form = StudentForm({
    "name": "Faruk",
    "age": "22"
})

if form.is_valid():
    print("Form is valid")
else:
    print("Form is invalid")
    print(form.errors)
```

Output:

```
Form is valid
```

If the age is `"abc"`, the form will be invalid because the value cannot be converted into a valid integer.

---

11. The `cleaned_data` Dictionary

`cleaned_data` contains validated and converted form values.

```
if form.is_valid():
    print(form.cleaned_data["name"])
    print(form.cleaned_data["age"])
```

Output:

```
Faruk
22
```

Common Type Conversions

Field	Python Type
`CharField`	`str`
`IntegerField`	`int`
`FloatField`	`float`
`DecimalField`	`decimal.Decimal`
`BooleanField`	`bool`
`DateField`	`datetime.date`
`DateTimeField`	`datetime.dateti`

	`me`

Rule: Read validated field values from `cleaned_data` after `is_valid()` returns `True`.

---

12. Complete Django Form Project

This example demonstrates a simple name form.

Project Structure

```
myproject/
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
└── contact/
    ├── forms.py
    ├── views.py
    ├── urls.py
    └── templates/
        └── contact/
            ├── name.html
            └── thanks.html
```

Step 1: Create the App

```
python manage.py startapp contact
```

Add the app to `INSTALLED_APPS` in `settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps...
    "contact",
]
```

Step 2: Create forms.py

```
from django import forms


class NameForm(forms.Form):
```

```
    your_name = forms.CharField(
        label="Your Name",
        min_length=2,
        max_length=100
    )
```

Step 3: Create views.py

```
from django.shortcuts import render, redirect

from .forms import NameForm


def get_name(request):
    if request.method == "POST":
        form = NameForm(request.POST)

        if form.is_valid():
            name = form.cleaned_data["your_name"]

            request.session["your_name"] = name

            return redirect("thanks")

    else:
        form = NameForm()

    return render(
        request,
        "contact/name.html",
        {"form": form}
    )


def thanks(request):
    name = request.session.get("your_name", "")

    return render(
        request,
        "contact/thanks.html",
```

```
            {"name": name}
        )
```

Step 4: Create contact/urls.py

```
from django.urls import path

from . import views


urlpatterns = [
    path(
        "your-name/",
        views.get_name,
        name="get_name"
    ),
    path(
        "thanks/",
        views.thanks,
        name="thanks"
    ),
]
```

Step 5: Configure Project URLs

In the project's `urls.py`:

```
from django.contrib import admin
from django.urls import path, include


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("contact.urls")),
]
```

Step 6: Create name.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Name Form</title>
```

```
</head>
<body>
    <h1>Enter Your Name</h1>

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

Step 7: Create thanks.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Thank You</title>
</head>
<body>
    <h1>Thank You!</h1>

    <p>Hello, {{ name }}!</p>
</body>
</html>
```

Step 8: Run the Development Server

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/your-name/
```

This example uses a Django session to carry the name to the success page. For a larger application, choose the appropriate persistence method based on the requirements.

---

13. Standard Form Handling Pattern

This is the standard pattern for handling a regular Django form.

```
from django.shortcuts import render, redirect

from .forms import MyForm


def my_view(request):
    if request.method == "POST":
        form = MyForm(request.POST)

        if form.is_valid():
            data = form.cleaned_data

            # Process or save validated data.
            return redirect("success")

    else:
        form = MyForm()

    return render(
        request,
        "my_template.html",
        {"form": form}
    )
```

Explanation

- `MyForm()` creates an unbound form.
- `MyForm(request.POST)` binds submitted data to the form.
- `is_valid()` runs validation.
- `cleaned_data` provides validated values.
- `redirect()` sends the browser to another URL.
- `render()` returns a response containing a rendered template.

If validation fails, the bound form is rendered again with the submitted values and validation errors.

---

14. CSRF Protection

CSRF stands for Cross-Site Request Forgery.

Django's CSRF protection helps prevent unauthorized cross-site requests from performing actions using a user's browser.

Template Example

```
<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Submit</button>
</form>
```

Important Points

1. Include `{% csrf_token %}` in applicable POST forms.
2. Django's CSRF middleware is enabled by default in a standard project configuration.
3. A CSRF token does not replace authentication or authorization.
4. Use HTTPS to protect sensitive data in transit.
5. Do not disable CSRF protection as a shortcut to fix form submission errors.

---

15. Contact Form Example

forms.py

```
from django import forms


class ContactForm(forms.Form):
    subject = forms.CharField(max_length=100)

    message = forms.CharField(
        widget=forms.Textarea
    )

    contact_email = forms.EmailField()

    urgent = forms.BooleanField(
        required=False
    )
```

views.py

```
from django.shortcuts import render, redirect

from .forms import ContactForm


def contact_view(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
            subject = form.cleaned_data["subject"]
            message = form.cleaned_data["message"]
            email = form.cleaned_data["contact_email"]
            urgent = form.cleaned_data["urgent"]

            # Save the data or send an email here.

            return redirect("contact_success")

    else:
        form = ContactForm()

    return render(
        request,
        "contact/contact.html",
        {"form": form}
    )


def contact_success(request):
    return render(
        request,
        "contact/success.html"
    )
```

contact.html

```
<h1>Contact Us</h1>
```

```
    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Send Message</button>
    </form>
```

Remember to define the corresponding URL patterns for `contact_view` and `contact_success`.

---

16. Form Rendering

Form rendering is the process of displaying Django form fields in an HTML template.

Method 1: `form`

```
    <form method="post">
        {% csrf_token %}

        {{ form }}

        <button type="submit">Submit</button>
    </form>
```

Method 2: `form.as_p`

```
    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Submit</button>
    </form>
```

Fields are rendered using paragraph-based markup.

Method 3: `form.as_div`

```
    <form method="post">
        {% csrf_token %}

        {{ form.as_div }}
```

```
        <button type="submit">Submit</button>
    </form>
```

Fields are rendered using `div` elements. This helper is available in modern Django versions; check the documentation for your installed version.

Method 4: `form.as_table`

```
    <form method="post">
        {% csrf_token %}

        <table>
            {{ form.as_table }}
        </table>

        <button type="submit">Submit</button>
    </form>
```

Important Points

- `form.as_p` : Paragraph-based rendering.
- `form.as_div` : Div-based rendering.
- `form.as_table` : Table-based rendering.
- `form.field` : Renders an individual field.
- `form.field.label_tag` : Renders a label.
- `form.field.errors` : Displays field errors.
- `form.non_field_errors` : Displays form-level errors.
- `form.hidden_fields` : Provides hidden fields.
- `form.visible_fields` : Provides visible fields.

Rendering helpers do not create the outer HTML `<form>` element or the submit button. Add those manually.

---

17. Rendering Individual Fields

Individual field rendering provides more control over form layout.

```
    <form method="post">
        {% csrf_token %}

        {{ form.non_field_errors }}
```

```
        <div>
            {{ form.subject.errors }}
            {{ form.subject.label_tag }}
            {{ form.subject }}
        </div>

        <div>
            {{ form.message.errors }}
            {{ form.message.label_tag }}
            {{ form.message }}
        </div>

        <div>
            {{ form.contact_email.errors }}
            {{ form.contact_email.label_tag }}
            {{ form.contact_email }}
        </div>

        <div>
            {{ form.urgent.errors }}
            {{ form.urgent.label_tag }}
            {{ form.urgent }}
        </div>

        <button type="submit">Submit</button>
    </form>
```

Important BoundField Attributes

When iterating over `form`, each item is a `BoundField`.

- `field.label`: The field's label.
- `field.label_tag`: The HTML label element.
- `field.errors`: Errors for that field.
- `field.help_text`: Additional instructions.
- `field.value`: The value used for rendering.
- `field.html_name`: The HTML input name.
- `field.id_for_label`: The ID associated with the label.
- `field.is_hidden`: Indicates whether the field is hidden.
- `field.field`: The underlying form field instance.

Loop Through Fields

```
{% for field in form %}
    <div class="fieldWrapper">
        {{ field.errors }}
        {{ field.label_tag }}
        {{ field }}

        {% if field.help_text %}
            <p>{{ field.help_text }}</p>
        {% endif %}
    </div>
{% endfor %}
```

---

18. Form Error Handling

Django provides field-level and form-level validation errors.

Example Form

```
from django import forms


class RegisterForm(forms.Form):
    username = forms.CharField(
        min_length=4,
        max_length=20
    )

    email = forms.EmailField()
```

Invalid Data

```
form = RegisterForm({
    "username": "ab",
    "email": "invalid-email"
})

print(form.is_valid())
print(form.errors)
```

Output:

```
False
```

The form is invalid because the username is too short and the email address does not have a valid format.

Display Field Errors

```
{{ form.username.errors }}
```

Display Non-field Errors

```
{{ form.non_field_errors }}
```

Custom Error Loop

```
{% if form.username.errors %}
    <ul class="errorlist">
        {% for error in form.username.errors %}
            <li>{{ error }}</li>
        {% endfor %}
    </ul>
{% endif %}
```

Displaying the bound form again after validation failure is often the simplest way to preserve user input and show errors.

---

19. Custom Validation

Custom validation allows developers to enforce application-specific rules beyond standard field validation.

A. `clean_fieldname()`

Use this method to validate an individual field.

```
from django import forms


class RegisterForm(forms.Form):
    username = forms.CharField(
        min_length=4,
        max_length=20
    )
```

```
        def clean_username(self):
            username = self.cleaned_data["username"]

            if username.lower() == "admin":
                raise forms.ValidationError(
                    "This username is not allowed."
                )

            return username
```

Django automatically calls `clean_username()` during form validation.

Important points:

- Use `self.cleaned_data` to retrieve the validated field value.
- Raise `forms.ValidationError` when the value violates a rule.
- Return the validated value when it is acceptable.

B. `clean()`

Use `clean()` for form-level validation, particularly when comparing multiple fields.

```
from django import forms


class RegisterForm(forms.Form):
    password = forms.CharField(
        widget=forms.PasswordInput
    )

    confirm_password = forms.CharField(
        widget=forms.PasswordInput
    )

    def clean(self):
        cleaned_data = super().clean()

        password = cleaned_data.get("password")
        confirm_password = cleaned_data.get(
            "confirm_password"
        )

        if (
            password
```

```
                and confirm_password
                and password != confirm_password
            ):
                raise forms.ValidationError(
                    "Passwords do not match."
                )

            return cleaned_data
```

C. `add_error()`

Use `add_error()` to associate an error with a specific field.

```
    def clean(self):
        cleaned_data = super().clean()

        password = cleaned_data.get("password")
        confirm_password = cleaned_data.get(
            "confirm_password"
        )

        if (
            password
            and confirm_password
            and password != confirm_password
        ):
            self.add_error(
                "confirm_password",
                "Passwords do not match."
            )

        return cleaned_data
```

This `clean()` method belongs inside a form class that defines both `password` and `confirm_password`.

D. Validator Functions

A validator can be reused across multiple forms.

```
    from django import forms
    from django.core.exceptions import ValidationError
```

```
    def validate_not_admin(value):
        if value.lower() == "admin":
            raise ValidationError(
                "The username 'admin' is not allowed."
            )


    class RegisterForm(forms.Form):
        username = forms.CharField(
            validators=[validate_not_admin]
        )
```

Use field-specific cleaning methods for field-specific rules, `clean()` for relationships between fields, and reusable validators for rules shared by multiple fields or forms.

---

20. Form vs ModelForm

Regular Form

A regular `Form` is useful when fields do not directly correspond to a database model or when the form is designed for custom processing.

```
    from django import forms


    class ProductForm(forms.Form):
        name = forms.CharField(max_length=100)

        price = forms.DecimalField(
            max_digits=10,
            decimal_places=2
        )
```

Model

```
    from django.db import models


    class Product(models.Model):
        name = models.CharField(max_length=100)

        price = models.DecimalField(
```

```
            max_digits=10,
            decimal_places=2
        )

        def __str__(self):
            return self.name
```

ModelForm

```
from django import forms
from .models import Product


class ProductModelForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "price"]
```

The `Meta` class specifies which model the form uses and which model fields are exposed.

Creating a Product

```
from django.shortcuts import render, redirect

from .forms import ProductModelForm


def create_product(request):
    if request.method == "POST":
        form = ProductModelForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("product_success")

    else:
        form = ProductModelForm()

    return render(
        request,
        "products/create.html",
```

```
            {"form": form}
        )
```

Differences

Form	ModelForm
Fields are generally defined manually	Fields can be generated from a model
Does not automatically save a model	Can save a model instance
Suitable for custom input processing	Suitable for creating and updating model objects
Requires custom persistence logic when saving is needed	Provides a convenient `save()` method

Important ModelForm Methods

`save()`

Validates are expected to have succeeded before saving.

```
if form.is_valid():
    product = form.save()
```

`save(commit=False)`

Creates a model instance without saving it to the database immediately.

```
if form.is_valid():
    product = form.save(commit=False)
    product.name = product.name.strip()
    product.save()
```

If the model has many-to-many fields, call `form.save_m2m()` after saving the instance when those relationships need to be saved.

Updating an Existing Object

```
product = Product.objects.get(pk=1)

form = ProductModelForm(
    request.POST or None,
    instance=product
)

if form.is_valid():
    form.save()
```

When using this pattern in a real view, distinguish GET and POST requests explicitly and handle the case where the requested object does not exist.

Security: Prefer explicit field allowlists such as `fields = ["name", "price"]` instead of exposing every model field.

Migration Commands

After changing model definitions:

```
python manage.py makemigrations
python manage.py migrate
```

---

21. Initial Data

The `initial` argument specifies values displayed when an unbound form is rendered.

```
from django import forms


class ProfileForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()


form = ProfileForm(
    initial={
        "name": "Faruk",
        "email": "faruk@example.com"
    }
)
```

Initial values are useful for:

- Default selections.
- Pre-populating profile forms.
- Showing default dates.
- Displaying suggested values.

Important: `initial` provides display defaults. It does not replace submitted values in a bound form and should not be treated as trusted input.

---

22. File Uploads in Django

File uploads allow users to submit documents, images, resumes, and other files through a web application.

A. Regular Form

forms.py

```
from django import forms


class DocumentForm(forms.Form):
```

```
    title = forms.CharField(max_length=100)
    document = forms.FileField()
```

Template

```
<form
    method="post"
    enctype="multipart/form-data"
>
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Upload</button>
</form>
```

View

```
from django.shortcuts import render, redirect

from .forms import DocumentForm


def upload_document(request):
    if request.method == "POST":
        form = DocumentForm(
            request.POST,
            request.FILES
        )

        if form.is_valid():
            uploaded_file = form.cleaned_data["document"]

            # Process or store the uploaded file.
            # A regular Form does not automatically save it.

            return redirect("upload_success")

    else:
        form = DocumentForm()
```

```
        return render(
            request,
            "documents/upload.html",
            {"form": form}
        )
```

B. Important File Upload Concepts

- `request.POST` : Contains ordinary submitted form data.
- `request.FILES` : Contains uploaded file objects.
- `multipart/form-data` : Required for standard HTML file uploads.
- `FileField` : Accepts uploaded files.
- `ImageField` : Validates image uploads and requires Pillow.
- `MEDIA_ROOT` : Defines the local directory for media storage.
- `MEDIA_URL` : Defines the URL prefix for serving media files.

C. File Storage Model

```
from django.db import models


class Document(models.Model):
    title = models.CharField(max_length=100)

    file = models.FileField(
        upload_to="documents/"
    )

    def __str__(self):
        return self.title
```

D. ModelForm for File Upload

```
from django import forms

from .models import Document


class DocumentModelForm(forms.ModelForm):
    class Meta:
```

```
            model = Document
            fields = ["title", "file"]
```

View:

```
from django.shortcuts import render, redirect

from .forms import DocumentModelForm


def upload_document(request):
    if request.method == "POST":
        form = DocumentModelForm(
            request.POST,
            request.FILES
        )

        if form.is_valid():
            form.save()
            return redirect("upload_document")

    else:
        form = DocumentModelForm()

    return render(
        request,
        "documents/upload.html",
        {"form": form}
    )
```

When a `ModelForm` is saved, Django uses the model's configured file storage to save the uploaded file and stores the file reference in the model.

E. Media Settings

In `settings.py`:

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

During development, media files can be served using:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # Your URL patterns...
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT
    )
```

This development helper is not intended for production media serving. Use an appropriate web server or storage service in production.

F. File Validation

```
import os

from django import forms

from .models import Document


class DocumentModelForm(forms.ModelForm):
    class Meta:
        model = Document
        fields = ["title", "file"]

    def clean_file(self):
        uploaded_file = self.cleaned_data["file"]

        if uploaded_file.size > 5 * 1024 * 1024:
            raise forms.ValidationError(
                "File size must not exceed 5 MB."
            )

        extension = os.path.splitext(
            uploaded_file.name
        )[1].lower()
```

```
            if extension != ".pdf":
                raise forms.ValidationError(
                    "Only PDF files are allowed."
                )

            return uploaded_file
```

This example checks file size and extension. These checks alone do not prove that a file contains a valid PDF.

File Upload Security

1. Validate file size.
2. Validate allowed file types.
3. Inspect file content when appropriate.
4. Use safe file storage and access permissions.
5. Avoid trusting the original filename.
6. Consider malware scanning for sensitive applications.
7. Avoid serving untrusted uploads as executable content.
8. Apply appropriate authorization checks before allowing downloads.

---

23. Hidden and Visible Fields

Django provides methods for working with hidden and visible fields.

```
<form method="post">
    {% csrf_token %}

    {% for hidden in form.hidden_fields %}
        {{ hidden }}
    {% endfor %}

    {% for field in form.visible_fields %}
        <div>
            {{ field.label_tag }}
            {{ field }}
            {{ field.errors }}
        </div>
    {% endfor %}
```

```
        <button type="submit">Submit</button>
    </form>
```

Important Points

- `form.hidden_fields` : Returns hidden fields.
- `form.visible_fields` : Returns visible fields.

Security Note: Hidden fields are still controlled by the client and can be modified.

Always validate hidden values on the server.

---

24. Reusable Form Templates

Custom form templates can help maintain consistent HTML markup across forms.

Example: Reusable Field Rendering

```
{% for field in form %}
    <div class="fieldWrapper">
        {{ field.errors }}
        {{ field.label_tag }}
        {{ field }}

        {% if field.help_text %}
            <p>{{ field.help_text }}</p>
        {% endif %}
    </div>
{% endfor %}
```

Using a Custom Form Template

In Django versions supporting the `Form.template_name` and `Form.render()` interfaces:

```
from django import forms


class MyForm(forms.Form):
    name = forms.CharField(max_length=100)

    template_name = "forms/custom_form.html"
```

A form can be rendered using its configured template or an explicitly supplied template:

```
form = MyForm()

rendered_form = form.render()
```

Ensure the template is accessible through Django's template loader. Check the documentation for your installed Django version when customizing the form renderer.

---

25. HTML5 Browser Validation

Django widgets can render HTML input types that support native browser validation.

Examples:

- `EmailField` typically uses an email input.
- `URLField` typically uses a URL input.
- `IntegerField` typically uses a number input.

Disabling Native Browser Validation

```
<form method="post" novalidate>
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Submit</button>
</form>
```

The `novalidate` attribute disables native browser form validation.

Django's server-side validation still runs when the submitted form is processed.

Important: Browser-side validation improves usability, but it must not replace server-side validation.

---

26. Formsets

A formset manages multiple instances of the same form together.

Common Uses

- Adding multiple students.
- Entering multiple products.
- Submitting several contact records.
- Processing multiple rows of data.

A. Create a Formset

forms.py

```
from django import forms
from django.forms import formset_factory


class ItemForm(forms.Form):
    name = forms.CharField(max_length=100)

    quantity = forms.IntegerField(
        min_value=1
    )


ItemFormSet = formset_factory(
    ItemForm,
    extra=3,
    max_num=5,
    validate_max=True
)
```

Here:

- `extra=3` : Displays three extra forms by default.
- `max_num=5` : Sets the maximum number of displayed forms.
- `validate_max=True` : Enforces the maximum during validation.

B. Formset Template

```
<h2>Add Multiple Items</h2>

<form method="post">
    {% csrf_token %}

    {{ formset.management_form }}

    {{ formset.non_form_errors }}

    {% for form in formset %}
        <div>
```

```
            {{ form.as_p }}
        </div>
    {% endfor %}

    <button type="submit">Submit All</button>
</form>
```

C. Process a Formset

```
from django.shortcuts import render, redirect

from .forms import ItemFormSet


def item_view(request):
    if request.method == "POST":
        formset = ItemFormSet(request.POST)

        if formset.is_valid():
            items = [
                form.cleaned_data
                for form in formset
                if form.cleaned_data
            ]

            # Process the validated items.
            return redirect("item_success")

    else:
        formset = ItemFormSet()

    return render(
        request,
        "items/formset.html",
        {"formset": formset}
    )
```

D. Why Is `management_form` Important?

`formset.management_form` supplies management data such as the number of submitted forms.

Django uses this information to construct and validate the submitted formset. Omitting it from the template normally causes formset validation to fail.

E. Formset Validation

```
if formset.is_valid():
    for form in formset:
        if form.cleaned_data:
            print(form.cleaned_data)
```

Always validate the formset before processing submitted values.

F. Formset vs ModelFormSet

Formset: Manages multiple instances of a regular `Form`.

ModelFormSet: Manages multiple `ModelForm` instances associated with database objects. It can create and update objects and, when configured appropriately, support deletion.

Example:

```
from django.forms import modelformset_factory

from .models import Student



StudentFormSet = modelformset_factory(
    Student,
    fields=["name", "email", "age"],
    extra=3
)
```

---

27. Django Form Security Best Practices

1. Validate user input on the server.
2. Include CSRF protection in applicable POST forms.
3. Apply authentication and authorization where necessary.
4. Use explicit field allowlists in `ModelForm`.
5. Use Django's authentication system for password management.
6. Validate uploaded file size, type, and content.
7. Use HTTPS to protect data in transit.

8. Avoid revealing sensitive information in error messages.
9. Enforce business rules and database constraints.
10. Never assume user input is trustworthy.
11. Use Django's ORM or parameterized database operations.
12. Validate hidden field values on the server.
13. Restrict access to private uploaded files.
14. Use appropriate rate limiting for sensitive endpoints.
15. Avoid storing plain-text passwords or sensitive secrets.

---

28. Common Django Form Mistakes

Mistake	Correct Approach
Processing data without validation	Call `is_valid()` first
Accessing values before validation	Use `cleaned_data` after successful validation
Omitting CSRF protection	Include `{% csrf_token %}` where required
Omitting `request`	Pass both `request`

`.FILES`	`.POST` and `request.FILES` for uploads
Omitting upload encoding	Use `multipart/form-data`
Not displaying validation errors	Re-render the bound form
Expecting a regular `Form` to save a model automatically	Use a `ModelForm` or implement persistence
Changing server state through GET	Use an appropriate state-changing method, commonly POST
Relying only on	Validate data on

browser validation	the server
Trusting hidden inputs	Validate hidden values on the server
Exposing every model field	Explicitly specify permitted fields
Omitting formset management data	Render `formset.management_form`
Trusting a file extension alone	Validate file content and storage requirements too
Forgetting migrations after model changes	Run `makemigrations` and `migrate`

---

29. Interview Questions and Answers

Q1. What are Django Forms?

Django Forms provide a framework for defining form fields, validating submitted data, converting values into Python types, and processing user input.

Q2. What is the difference between `Form` and `ModelForm`?

`Form` generally requires fields to be defined manually. `ModelForm` can generate fields from a model and provides a convenient `save()` method for model instances.

Q3. What is a bound form?

A bound form is associated with submitted or otherwise supplied data.

Q4. What is an unbound form?

An unbound form has not been associated with submitted data.

Q5. What does `is_valid()` do?

It runs form validation and returns `True` when the form passes validation.

Q6. What is `cleaned_data`?

It is a dictionary containing validated and converted form values.

Q7. What is `clean_fieldname()`?

It is a method used to perform custom validation on a specific field.

Q8. What is `clean()`?

It performs form-level validation, often including checks involving multiple fields.

Q9. What is a widget?

A widget controls how a form field is rendered as an HTML input element.

Q10. Why is `csrf_token` used?

It supplies a token used by Django's CSRF protection mechanism to help prevent cross-site request forgery.

Q11. What is the difference between `request.POST` and `request.FILES`?

`request.POST` contains submitted form data, while `request.FILES` contains uploaded files.

Q12. What is the purpose of a Django view?

A view receives an HTTP request, executes application logic, processes input, and returns an HTTP response.

Q13. What is `initial`?

It specifies initial display values for form fields.

Q14. Why should GET requests not change database state?

GET is intended primarily for retrieving data. State-changing operations should use an appropriate method, commonly POST.

Q15. What is a formset?

A formset manages multiple instances of the same form.

Q16. What is a ModelFormSet?

A ModelFormSet manages multiple model-backed forms and supports operations such as creating and updating model instances.

Q17. What is the purpose of `form.errors`?

It provides validation errors associated with the form's fields and form-level validation.

Q18. What is the difference between `clean()` and `clean_fieldname()`?

`clean_fieldname()` validates an individual field, while `clean()` performs form-level validation.

Q19. What does `save(commit=False)` do?

It creates a model instance without immediately saving it to the database.

Q20. Why is `request.FILES` necessary?

It provides access to uploaded files in a request.

Q21. Why is `management_form` required in a formset?

It provides management data that Django needs to construct and validate the submitted formset.

Q22. What is the difference between field errors and non-field errors?

Field errors are associated with specific fields. Non-field errors apply to the form as a whole.

---

30. Practice Problems

Problem 1: Student Form

Create a form containing:

- Name
- Email

- Age between 16 and 60

Requirements:

- Validate every field.
- Display appropriate errors.
- Preserve input when validation fails.

Problem 2: Custom Username Validation

Create a registration form with these rules:

- Minimum length: 5 characters.
- Maximum length: 20 characters.
- The username `admin` must not be accepted.

Use `clean_username()`.

Problem 3: Contact Form

Create a contact form containing:

- Subject
- Message
- Email
- Optional urgent checkbox

Requirements:

- Use a `Textarea` widget for the message.
- Validate the email.
- Display field errors.

Problem 4: ModelForm

Create a `Book` model with:

- Title
- Author
- Publication year

Create a `ModelForm` and save a valid book to the database.

Problem 5: File Upload

Create a PDF upload form.

Requirements:

- Set a maximum file size.
- Check the file type.

- Save the uploaded file.
- Handle validation errors.

Problem 6: Password Confirmation

Create a registration form containing:

- Username
- Password
- Confirm password

Use `clean()` to ensure both passwords match. For a real registration system, use Django's authentication tools rather than saving raw passwords yourself.

Problem 7: Product Creation

Create a product form containing:

- Name
- Price
- Quantity
- Availability

Use `ModelForm` to save the product.

Problem 8: Bulk Student Entry

Create a student formset.

Requirements:

- Display three forms initially.
- Validate every submitted form.
- Display formset-level errors.
- Process only validated data.

Problem 9: Profile Update

Create a profile update page that:

- Loads an existing model instance.
- Displays existing values.
- Validates submitted changes.
- Saves the updated instance.

Problem 10: Custom Form Rendering

Create a form template that:

- Displays each label and input separately.

- Shows field errors.
- Shows help text.
- Uses CSS classes.
- Includes CSRF protection.

---

31. Quick Revision Notes

- `forms.Form` : Creates a regular form.
- `forms.ModelForm` : Creates a model-based form.
- `CharField` : Accepts text.
- `EmailField` : Validates email-like input.
- `IntegerField` : Validates and converts integer data.
- `DecimalField` : Handles decimal values.
- `widget` : Defines the HTML representation of a field.
- `initial` : Defines initial display values.
- `request.POST` : Contains submitted form data.
- `request.FILES` : Contains uploaded files.
- `form.is_valid()` : Runs form validation.
- `form.cleaned_data` : Contains validated Python values.
- `form.errors` : Contains validation errors.
- `form.non_field_errors()` : Provides form-level errors.
- `clean_<fieldname>()` : Performs field-specific validation.
- `clean()` : Performs form-level validation.
- `add_error()` : Associates a validation error with a field or the form.
- `form.save()` : Saves a model instance through a ModelForm.
- `form.save(commit=False)` : Creates an unsaved model instance.
- `form.hidden_fields()` : Returns hidden fields.
- `form.visible_fields()` : Returns visible fields.
- `{% csrf_token %}` : Renders the CSRF token.
- `formset.management_form` : Provides management data for a formset.
- `multipart/form-data` : Enables standard HTML file uploads.
- `MEDIA_ROOT` : Defines the local media storage directory.
- `MEDIA_URL` : Defines the media URL prefix.

---

32. The Most Important Django Form Pattern

Memorize this pattern first:

```
from django.shortcuts import render, redirect

from .forms import MyForm


def my_view(request):
    if request.method == "POST":
        form = MyForm(request.POST)

        if form.is_valid():
            data = form.cleaned_data

            # Process or save validated data.
            return redirect("success")

    else:
        form = MyForm()

    return render(
        request,
        "my_template.html",
        {"form": form}
    )
```

For a file-upload form, bind both ordinary form data and uploaded files:

```
form = MyForm(
    request.POST,
    request.FILES
)
```

For a `ModelForm`, save the model instance only after successful validation:

```
if form.is_valid():
    form.save()
```

For a formset, render its management form and validate the formset before processing its data:

```
{{ formset.management_form }}
```

```
if formset.is_valid():
    # Process validated forms.
    pass
```

Conclusion

To master Django Forms, learn the concepts in this order:

1. Form classes and field types.
2. GET and POST requests.
3. Bound forms and unbound forms.
4. `is_valid()`, `cleaned_data`, and errors.
5. Widgets and form rendering.
6. Custom validation.
7. `ModelForm` and database operations.
8. File uploads and file validation.
9. Formsets and ModelFormSets.
10. Security and real-world project implementation.

Final recommendation: Practice by building a registration form, contact form, student management system, product creation page, and document upload system. These projects will help you understand how Django Forms are used in real-world backend development.